Traps Lab - Ayaan Arif Nadeem - 30546

RISC-V assembly

output

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ call
12 13
HE110 World
```

BACKTRACE

1)Added def in defs.h

```
// printk.c
int             printk(char*, ...) __attribute__ ((format (printf, 1, 2)));
void            panic(char*) __attribute__((noreturn));
void            printkinit(void);
void            backtrace(void);
```

2)Added func to allow reading the s0 register in riscv.h

```
//way to read the s0 register
static inline uint64
r_fp()
{
  uint64 x;
  asm volatile("mv %0, s0" : "=r" (x) );
  return x;
}
```

3)implemented func in printk.c

```
void
backtrace(void)
{
  printk("backtrace:\n");
  uint64 fp = r_fp();
  uint64 top = PGROUNDUP(fp);

  while(fp < top) {
    uint64 ra = *(uint64*)(fp - 8);
    printk("%p\n",(void*)ra);
    fp = *(uint64*)(fp - 16);
  }
}
```

4)added func call to sys_pause in sysproc.c

```
uint64
sys_pause(void)
{
  int n;
  uint ticks0;

  backtrace();

  argint(0, &n);
  if (n < 0)
    n = 0;
  acquire(&tickslock);
  ticks0 = ticks;
  while (ticks - ticks0 < n) {
    if (killed(myproc())) {
      release(&tickslock);
      return -1;
    }
    sleep_prepare(&ticks);
    release(&tickslock);
    sleep();
    acquire(&tickslock);
  }
  release(&tickslock);
  return 0;
}
```

5)Added func call to panic in printk.c

```
void
panic(char *s)
{
  panicking = 1;
  printk("panic: ");
  printk("%s\n", s);
  panicked = 1; // freeze uart output from other CPUs
  backtrace();
  for (;;)
    ;
}
```

output

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ bttest
backtrace:
0x0000000080001ea6
0x0000000080001d82
0x0000000080001b02
```

```
root@DESKTOP-RBTBS7K:~/xv6-postmtlabs/xv6-labs-2026# addr2line -e kernel/kernel
0x0000000080001ea6
./kernel/sysproc.c:75
0x0000000080001d82
./kernel/syscall.c:146 (discriminator 1)
0x0000000080001b02
./kernel/trap.c:81
```

ALARM

1)Added $U/_alarmtest to the UPROGS list in Makefile

```
UPROGS=\
	$U/_cat\
	$U/_echo\
	$U/_forktest\
	$U/_grep\
	$U/_init\
	$U/_kill\
	$U/_ln\
	$U/_ls\
	$U/_mkdir\
	$U/_rm\
	$U/_sh\
	$U/_stressfs\
	$U/_usertests\
	$U/_grind\
	$U/_wc\
	$U/_zombie\
	$U/_logstress\
	$U/_forphan\
	$U/_dorphan\
	$U/_sync\
	$U/_alarmtest
```

2)Added system calls in user.h

```
int sigalarm(int ticks, void (*handler)());
int sigreturn(void);
```

3)added assembly stubs in usys.pl

```
entry("sigalarm");
entry("sigreturn");
```

4)assigned unique func call numbers to funcs in syscall.h

```
#define SYS_sigalarm 23
#define SYS_sigreturn 24
```

5) added func prototypes and func array mapping in syscall.c

```
extern uint64 sys_sigalarm(void);
extern uint64 sys_sigreturn(void);
```

```
[SYS_sigalarm] =   sys_sigalarm,
[SYS_sigreturn] =  sys_sigreturn,
// clang-format on
```

TEST 0

a) Added alarm field to process structure in proc.h in struct proc

```
int alarm_interval;          // Alarm interval in ticks
uint64 alarm_handler;        // Address of the user-space handler function
int ticks_count;             // How many ticks have passed since the last alarm
```

b) Initialized new fields in proc.c in func allocproc so that they default to 0 for new process

```
p->alarm_interval = 0;
p->alarm_handler = 0;
p->ticks_count = 0;
```

c) Add func definition to sysproc.c

```
uint64
sys_sigalarm(void)
{
  int interval;
  uint64 handler;
  struct proc *p = myproc();

  argint(0, &interval);
  argaddr(1, &handler);

  p->alarm_interval = interval;
  p->alarm_handler = handler;
  p->ticks_count = 0;

  return 0;
}
```

d) Modified timer interrupts handler in traps.c to of usertrap()

```
// give up the CPU if this is a timer interrupt.
if(which_dev == 2) {
  if(p->alarm_interval > 0) {
    p->ticks_count++;
    if(p->ticks_count == p->alarm_interval) {
      p->trapframe->epc = p->alarm_handler;
      p->ticks_count = 0;
    }
  }
  yield();
}

prepare_return();
```

TEST 1

a) two more fields to the end of struct proc in proc.h

```
struct trapframe saved_trapframe; // Backup of the trapframe
int alarm_active;                 // Flag to track if the handler is currently running
```

b) initialize in proc.c in allocproc()

```
p->alarm_active = 0;
```

c) update usertrap() in trap.c

```
// give up the CPU if this is a timer interrupt.
if(which_dev == 2) {
  if(p->alarm_interval > 0) {
    p->ticks_count++;
    if(p->ticks_count == p->alarm_interval && p->alarm_active == 0) {
      // Save the current trapframe
      p->saved_trapframe = *(p->trapframe);

      // Mark alarm as active and jump to handler
      p->alarm_active = 1;
      p->trapframe->epc = p->alarm_handler;
      p->ticks_count = 0;
    }
  }
  yield();
}
```

d) Implement sigreturn()

```
uint64
sys_sigreturn(void)
{
  struct proc *p = myproc();

  // Restore the saved trapframe
  *(p->trapframe) = p->saved_trapframe;

  // Reset the alarm active flag so future alarms can trigger
  p->alarm_active = 0;

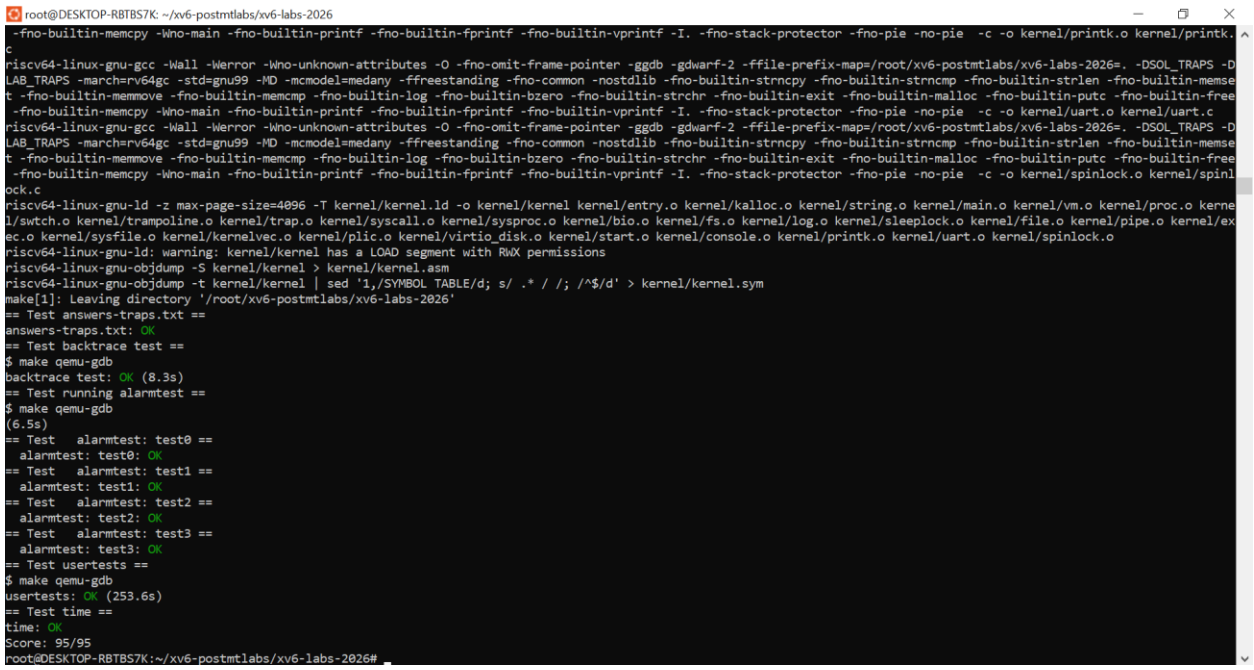
  // Return the original a0 register value
  return p->trapframe->a0;
}
```

Output

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ alarmtest
test0 start
....alarm!
test0 passed
test1 start
.alarm!
.alarm!
.alarm!
alarm!
.alarm!
.alarm!
.alarm!
.alarm!
alarm!
.alarm!
test1 passed
test2 start
....alarm!
test2 passed
test3 start
test3 passed
$
```

Make Grade


root@DESKTOP-RBTBS7K: ~/xv6-postmtlabs/xv6-labs-2026
-fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/printk.o kernel/printk.c
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/root/xv6-postmtlabs/xv6-labs-2026=. -DSOL_TRAPS -DLAB_TRAPS -march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/uart.o kernel/uart.c
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/root/xv6-postmtlabs/xv6-labs-2026=. -DSOL_TRAPS -DLAB_TRAPS -march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/spinlock.o kernel/spinlock.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/entry.o kernel/kalloc.o kernel/string.o kernel/main.o kernel/vm.o kernel/proc.o kernel/swtch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o kernel/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o kernel/pipe.o kernel/exec.o kernel/sysfile.o kernel/kernelvec.o kernel/plic.o kernel/virtio_disk.o kernel/start.o kernel/console.o kernel/printk.o kernel/uart.o kernel/spinlock.o
riscv64-linux-gnu-ld: warning: kernel/kernel has a LOAD segment with RWX permissions
riscv64-linux-gnu-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-linux-gnu-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kernel/kernel.sym
make[1]: Leaving directory '/root/xv6-postmtlabs/xv6-labs-2026'
== Test answers-traps.txt ==
answers-traps.txt: OK
== Test backtrace test ==
$ make qemu-gdb
backtrace test: OK (8.3s)
== Test running alarmtest ==
$ make qemu-gdb
(6.5s)
== Test alarmtest: test0 ==
alarmtest: test0: OK
== Test alarmtest: test1 ==
alarmtest: test1: OK
== Test alarmtest: test2 ==
alarmtest: test2: OK
== Test alarmtest: test3 ==
alarmtest: test3: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (253.6s)
== Test time ==
time: OK
Score: 95/95
root@DESKTOP-RBTBS7K:~/xv6-postmtlabs/xv6-labs-2026#